Software Implementation and Testing Document

For

Group 10

Version 2.0

Authors:
Kirra Orndorff
Alexander Jubran
Lauren Chang
Riley Galpin
Michael Greenberg

1. Programming Languages (5 points)

C#: This is our primary language for application logic. We chose C# because it is the native language for the .NET MAUI framework, allowing us to share a single codebase across multiple platforms while maintaining high performance.

XAML: Used in conjunction with C# within the .NET MAUI framework to define the user interface. It allows for a clean separation between the UI design and the underlying business logic (MVVM pattern).

2. Platforms, APIs, Databases, and other technologies used (5 points)

Platform: .NET MAUI: We are using this framework to build a cross-platform application that can run as a web application while retaining the ability to deploy to mobile devices in the future. We will use .NET SDK version 8.0.

Weather API (weatherAPI: This API is used to retrieve real-time environmental data including temperature, wind speed, and humidity to power our clothing recommendation engine.

.NET Geolocation: To be used in conjunction with the WeatherAPI. It provides APIs to retrieve the device's current geolocation coordinates.

Google Maps API: Integrated to allow users to view, create, and save custom running routes with specific terrain data.

OpenRouteService: for routing API

OpenStreetMapAPI: contributors for map data

MapLibreAPI: for the mapping library

Turf.jsAPI: for geospatial calculations

Google CalendarAPI: for weekly planner and past workout log.

Local Storage (.NET MAUI Built-in): We are utilizing the framework's internal storage capabilities to ensure user privacy and provide offline access to saved routes and shoe mileage.

UraniumUI.Material & Community Toolkit.Mvvm: These libraries are used to enhance the UI experience and implement the Model-View-ViewModel architecture for better code maintainability.

3. Execution-based Functional Testing (10 points)

The Manual Weather Override requirement was tested by taking our computers offline and seeing if the manual weather input screen would display upon opening the app, as compared to just the main home page if the computer was connected to wifi. The Clothing Recommendation Engine was tested by manually changing the values connected to the input variables in the recommendation engine and checking the output to see if the output accurately reflected the recommendation system based on the temperature, harsh conditions, etc. Additionally, as we implemented some functionality to the screens, buttons were checked to confirm successful additions and deletions from the database including checking edge cases which may lead to null errors due to incorrect or missing input types .

4. Execution-based Non-Functional Testing (10 points)

The two non-functional requirements tested during this increment were the Privacy-First Architecture and Offline Functionality requirements. The privacy-first architecture requirement was implemented and tested by using only local data via .NET MAUI's built in storage platform to prevent data being shared or leaked. The offline functionality was tested by creating a manual weather input screen that appears if the user is not on wifi, so that they can still access all of the main features of the app even without the internet.

5. Non-Execution-based Testing (10 points)

Since this increment was relatively front-end heavy, a majority of the code inspections/reviews was checking the consistency of styles between pages. Adjustments were made to the layout, text size/font, and colors in order to make sure that each page of the app all matched the same central theme. Buttons and detail pages were also tested to make sure that they correctly navigated to the desired page.